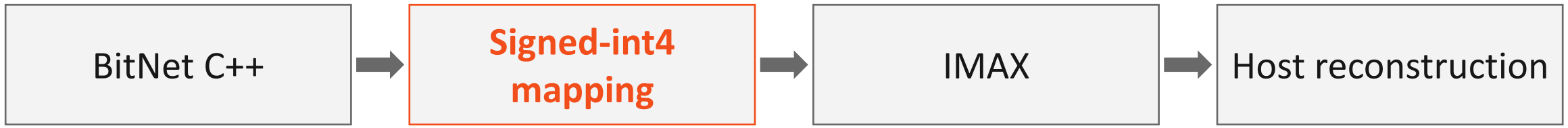
Implementation and Evaluation of BitNet Inference on a CGLA by Signed-Int4 Instructions

Takuto Ando and Yasuhiko Nakashima

Nara Institute of Science and Technology (NAIST)



Why Low-Bit Arithmetic?

BitNet b1.58

- Ternary weights reduce model-weight storage.
- Every generated token still requires dot products and data movement.
- The existing int8 path uses eight bits for each ternary weight.

$$w \in \{-1, 0, +1\}$$

$$y = \Sigma_i \, w_i \, a_i$$

Ternary weight $\times$ int8 activation
32-bit integer accumulation

The arithmetic format must fit the programmable instruction path.

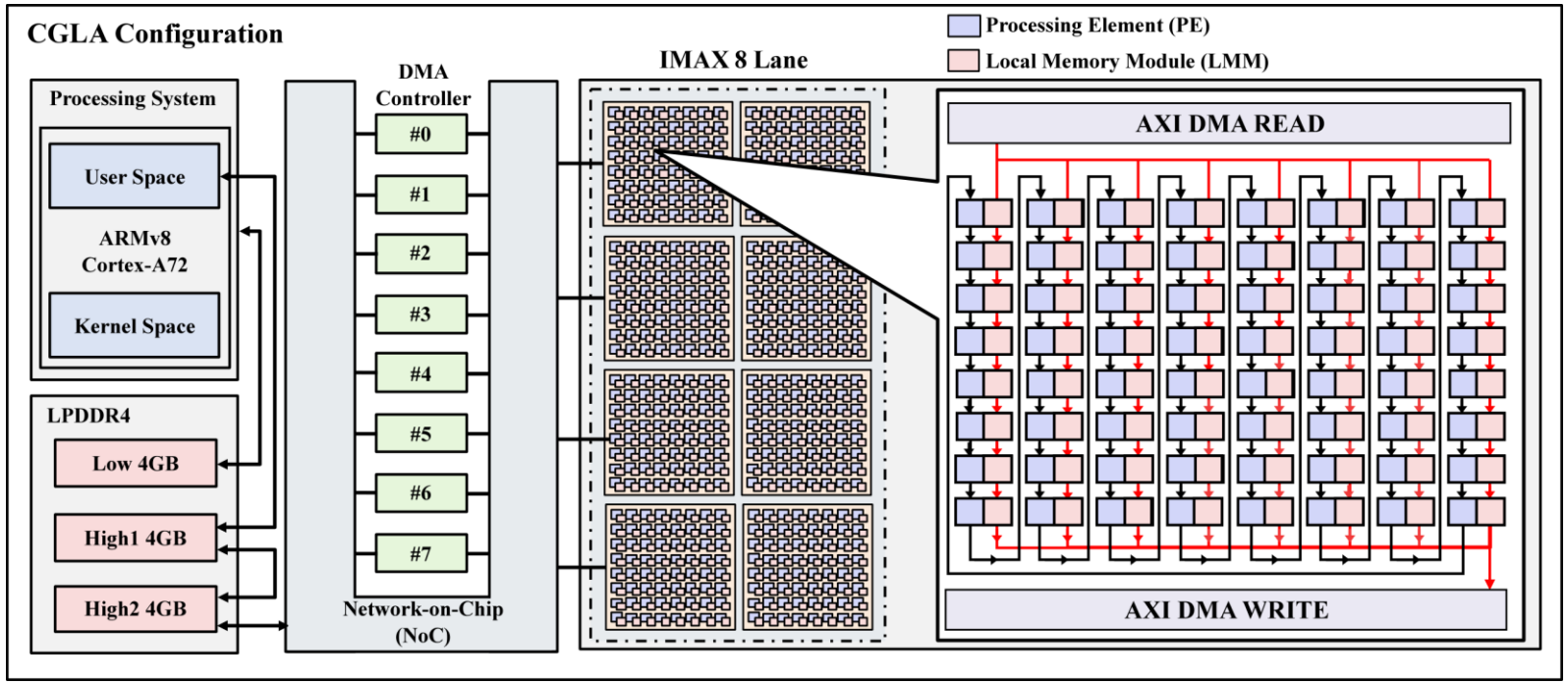
Objective and Evaluation Boundaries

Map ternary–int8 dot products to a reusable signed-int4 instruction

Boundary	What is evaluated
Instruction	One signed-int4 stream, including staging and drain
Exact mapping	Two activation streams and sparse correction
Runtime	BitNet C++ decode with one selected offloaded call

Keep instruction cost, layer projections, and decode timing separate.

IMAX Configuration



Host / accelerator

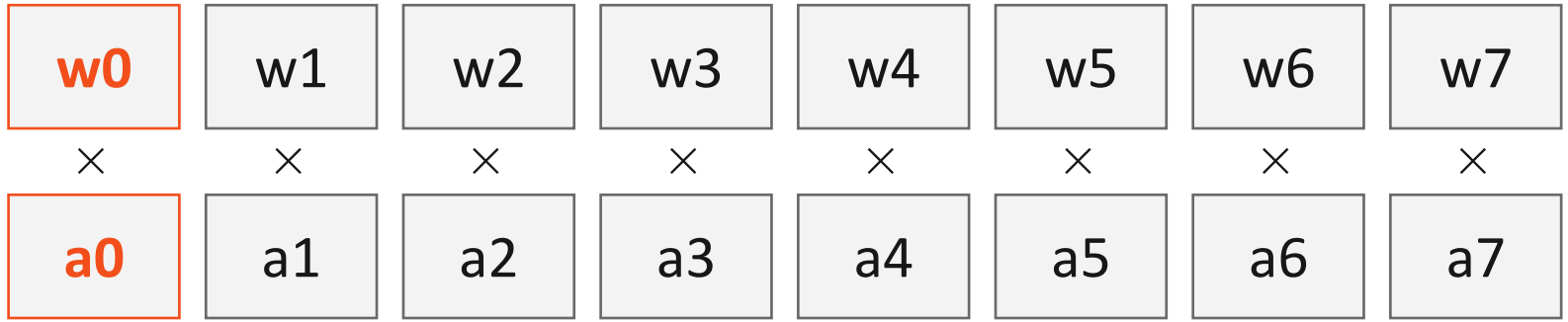
- ARM controls DMA and kernel launch.
- PEs and LMMs form a linear pipeline.
- The compiler assigns operations to ordered rows.

CPU-Grounded Linear Array (CGLA) · a programmable ASIC architecture

FPGA prototype · AMD Versal VPK180 · 145 MHz · 64-stage pipeline

Signed-Int4 Instruction

Eight signed-int4 products per 32-bit lane



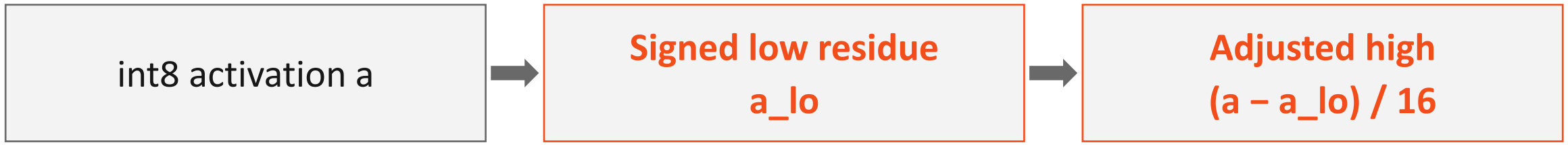
Each field
–8 to +7
Signed int4

$$d' = d + \Sigma_j \, \mathrm{sext}_4(w_j) \cdot \mathrm{sext}_4(a_j)$$

One 64-bit instruction updates two 32-bit lanes independently.

Representing an Int8 Activation

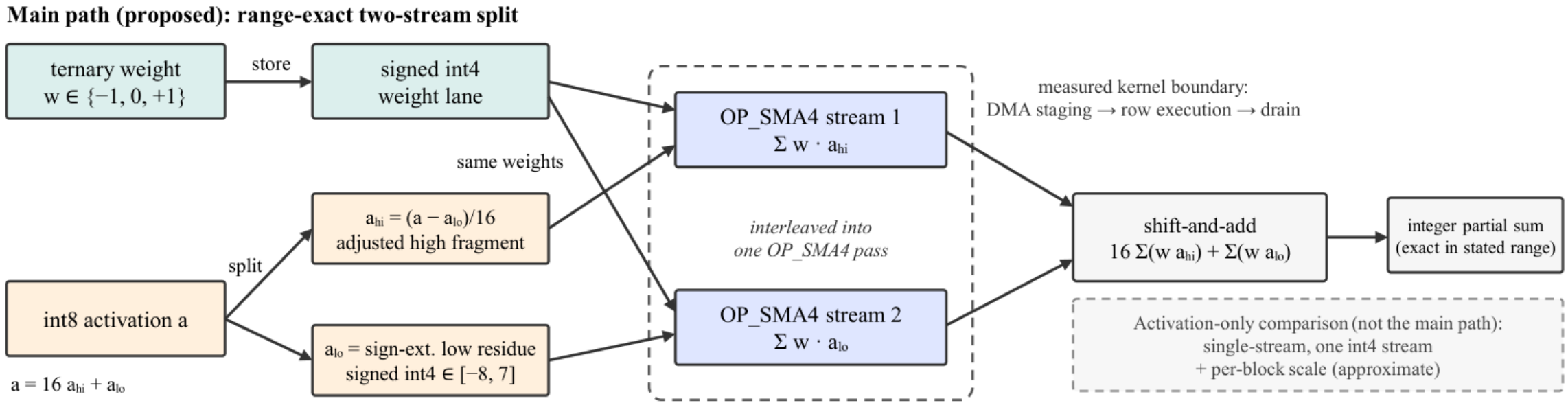
$$a = 16a_hi + a_lo$$



Activation range	Signed-int4 representation
−128 to 119	Both fragments lie in [−8, 7]
120 to 127	Adjusted high = 8 → encode as −8 and correct

An exact ternary–int8 product requires two int4 fragment products.

Exact Dot-Product Reconstruction

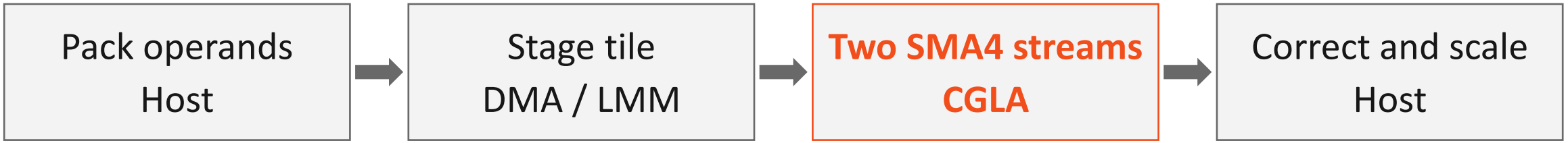


Exact for $-128 \le a \le 119$: both fragments fit signed int4 $[-8, 7]$, so the two int4 streams reconstruct the original int8 activation and its integer dot product exactly; $a \in [120, 127]$ uses sparse host correction for exact reconstruction.

$$y = 16s_hi + s_lo + c, \quad c = 256\,\Sigma\, w \text{ for activations } 120\text{–}127$$

Sparse correction restores the full int8 range before block scaling.

BitNet C++ Offload Path



Fused-output variant

CGLA forms s_lo + 16s_hi before output transfer.

Host retains sparse correction and scaling.

Separate sums → one combined partial

CPYOUT 1.56 → 0.78 MB

Reuse the existing DMA, synchronization, and row-scheduling interface.

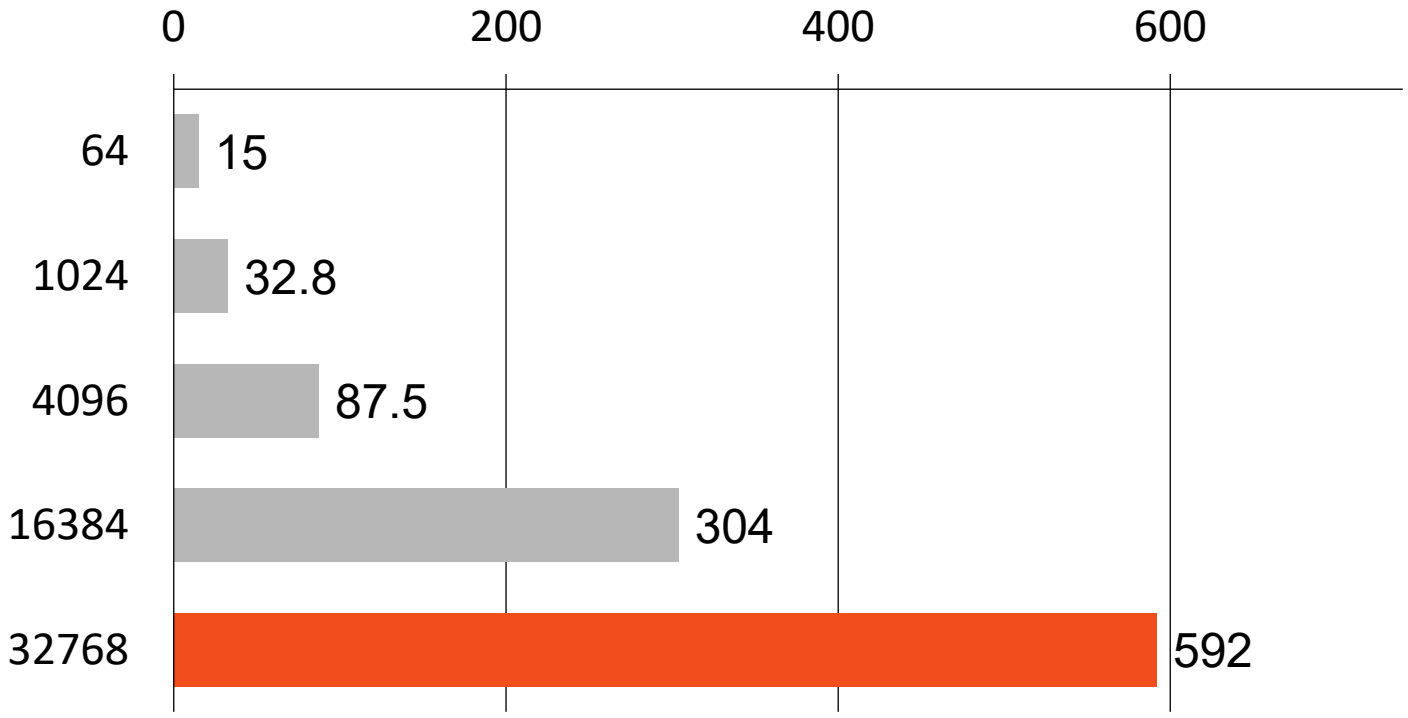
Experimental Setup

Item	Setting
Prototype	145 MHz FPGA, cacheable DDR path
Model / runtime	BitNet b1.58 2B-4T / BitNet C++
Integrated run	Two host threads, 19 generated tokens, five paired runs
Selected offload	2560 × 2560 projection, one call after skipping 420
Timing scope	Decode evaluation after warm-up and prompt evaluation
ASIC context	28 nm, 840 MHz frequency-only projection

The integrated run exercises the offload path inside the existing runtime.

Signed-Int4 Instruction Timing

FPGA interval (µs) by loaded 32-bit words



At 32768 words

2.26 ns / int4 product

FPGA measurement

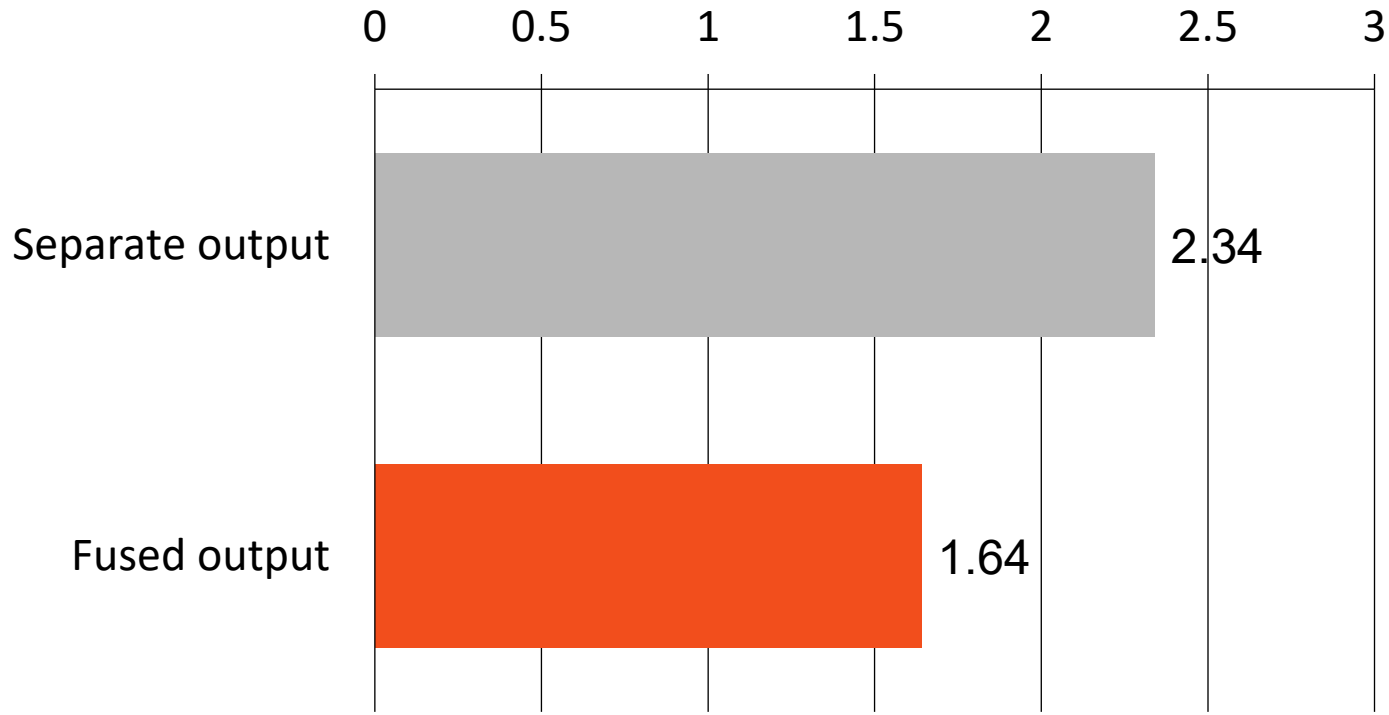
0.390 ns / int4 product

840 MHz projection

Measured intervals include input staging, execution, and output drain.

Reducing Output Transfer

Output-drain time (ms)



CPYOUT volume
1.56 → 0.78 MB

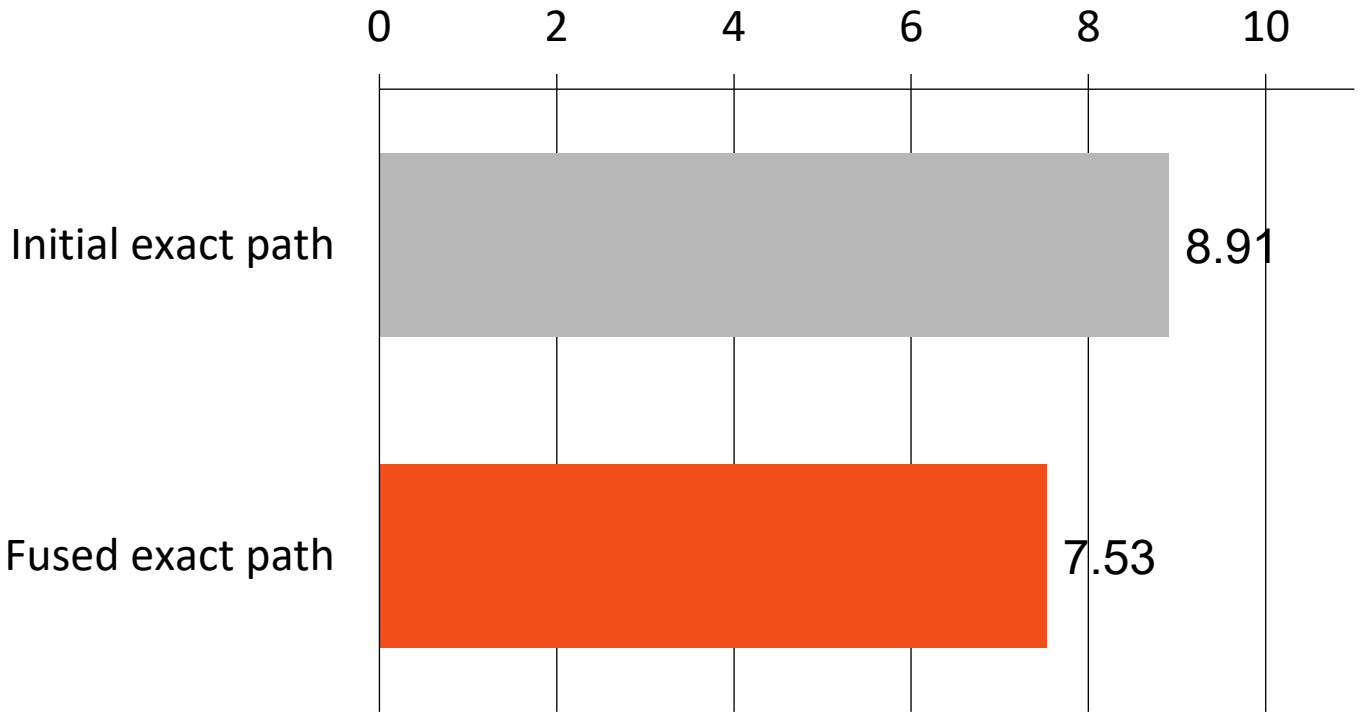
Active interval
17.0 → 16.4 ms

Lower drain time
in all five pairs

Input staging and array execution remain after output fusion.

Integrated Decode Measurement

19-token decode time (s)



2.52 tokens/s

One offloaded call
16.3 ms active interval
8.88 ms staging
5.82 ms execution
1.64 ms drain

Approximately 7.51 s remains on the host in the fused run.

Five-run medians; initial and fused configurations differ beyond output fusion alone.

Exact and Approximate Activation Paths

Path	Path time (s)	Normalized dot error
Exact two-stream	0.595	0
Max-abs	0.359	0.102
Power-of-two	0.387	0.0979
Asymmetric affine	0.432	0.0821
Activation-MSE	1.97	0.0942

One stream reduces arithmetic but introduces partial-dot error.

One selected projection; 15 timing runs and three verification runs. No downstream quality claim.

Throughput and Energy Context

Implementation	tokens/s	Estimated J/token
CGLA fused path	2.52	0.258
Xeon W5-2465X	8.66	7.59
RTX 4090 I2_S	8.17	79.56
RTX 4090 W2A8 reference	406.5	1.599

I2_S is format matched. W2A8 uses a separate PyTorch CUDA-graph path.
CGLA energy uses phase accounting and the 28 nm model.

The lowest modeled-energy row also has the lowest decode throughput.

Manuscript power assumptions; host-dominated one-call run does not establish full-model offload savings.

What Limits the Current Mapping?

From instruction support to full-model offload

- Exact int8 reconstruction still requires two fragment streams.
- The current integrated run offloads one selected call.
- Host processing dominates the measured decode interval.
- ASIC frequency and power values are model-based estimates.

Next steps are host/DMA optimization and wider runtime coverage.

Summary

BitNet arithmetic on a programmable CGLA

- Implemented OP_SMA4 as a reusable signed-int4 instruction.
- Mapped ternary–int8 dots using two streams and sparse correction.
- Measured 2.52 tokens/s with one offloaded call in BitNet C++.
- Separated instruction timing, projected layer cost, and runtime overhead.



Thanks!

Do you have any questions?

Takuto Ando · Nara Institute of Science and Technology

Backup · Layer-Time Projection

Quantity	Value / interpretation
Projection shape	m = 6912, k = 2560
Original ternary–int8 MACs	1.77×10^7
One int4 stream at 145 MHz	Approximately 40 ms, estimated
Exact two-stream mapping	Approximately 80 ms at 145 MHz
840 MHz context	Approximately 14 ms, frequency-only projection

These layer values are estimates, not directly timed full-layer invocations.